WORKSHEET-02: UNIT II

Research Methodology in Computer Science

Dialectic, Arguments, Proofs, Deduction, Induction, Models and Evaluation

Name:	Krish Aggarwal	**Roll No.:**	590014301
Section:	B1_CCVT	**Date:**	23/10/2026

LEARN & REMEMBER

Purpose of this worksheet

This worksheet helps you understand and apply the major concepts of Unit II. Read the short explanation, complete the examples, and then attempt the practice questions.

CLASSROOM ACTIVITY

Unit II in one line

Claim → Argument → Challenge → Proof/Evidence → Model → Evaluation

UNIT II — LEARNING MAP

Tick each topic after completing it.

#	Topic	Done	Confidence (1-5)
1	Dialectic of Research in CS	✓	4
2	Models of Argument	✓	5
3	Proof Methods	✓	4
4	Proof by Demonstration	✓	5
5	Empirical Proofs	✓	4
6	Mathematical Proofs	✓	4
7	Deduction and Induction	✓	5
8	Theoretical Models	✓	4
9	Algorithmic Approaches	✓	4
10	Software Engineering Approaches	✓	4
11	Mathematical Modelling	✓	4
12	Performance Evaluation	✓	5

PRACTICE ZONE — QUICK WARM-UP

1. General → specific reasoning is called:

Deduction

2. Specific observations → general rule is called:

Induction

3. Reasoning toward the “best explanation” is called:

Abduction

4. A claim becomes stronger when it is:

tested against counter-arguments and supported by evidence

PART A — DIALECTIC OF RESEARCH

Using claims, counterclaims and evidence to refine research

LEARN & REMEMBER

Dialectic is structured reasoning through opposing or different views. In Computer Science research, the process can be understood as:

Claim → Challenge → Evidence → Refinement

It helps researchers:

- identify weaknesses in an idea,
- consider alternative viewpoints,
- reduce premature conclusions,
- refine vague research ideas,
- strengthen research arguments.

A1. Dialectical Process

1. Identify a claim or hypothesis. 2. Challenge it using counter-arguments or experiments. 3. Examine evidence and logic. 4. Refine the claim. 5. Arrive at a stronger explanation or solution.

PRACTICE ZONE

Claim: *"Greedy algorithms always produce optimal results."*

1. What is the claim?

Greedy algorithms always give the best (optimal) solution for every problem.

2. Give a counterexample or situation where it may fail.

Coin change with coins {1, 3, 4} and amount 6: greedy picks 4+1+1 (3 coins), but the optimal answer is 3+3 (2 coins).

3. Write a more accurate version of the claim.

Greedy algorithms give optimal results only for problems with the greedy-choice property and optimal substructure (e.g., Huffman coding, Kruskal's algorithm).

4. What evidence could support the revised claim?

A formal proof (exchange argument) of the greedy-choice property, plus tests that compare greedy output with a brute-force or dynamic-programming answer on many inputs.

A2. Dialectic and Problem Formulation

Choose one: AI in Education | Smart Traffic | Robotic Farming | IoT Security

Chosen topic: AI in Education

Initial idea	Use AI to give each student personalised practice questions.
Possible claim	AI-based personalised practice improves students' exam scores.
Counter-argument	Scores may rise only because of extra attention or novelty, not the AI. Students also differ a lot, and the tool may be biased or misuse student data.
Evidence required	Pre-test and post-test scores of an AI group and a comparison group of similar students, collected over the same period.
Refined research problem	Does an adaptive AI quiz tool raise the programming test scores of first-year students more than a standard online quiz over one semester?

PART B — MODELS OF ARGUMENT

Deductive, inductive and abductive reasoning

LEARN & REMEMBER

There are three important reasoning models:

	Deductive	**Inductive**	**Abductive**
Direction	General → Specific	Specific → General	Observation → Explanation
Conclusion	Certain if premises are true	Probable	Plausible
Typical CS use	Proof	ML / data analysis	Debugging

B1. Deductive Reasoning

Example: All Dijkstra results are optimal when its assumptions are satisfied. Graph G satisfies those assumptions. Therefore, the shortest-path result for G is optimal.

PRACTICE ZONE — CREATE YOUR OWN DEDUCTIVE ARGUMENT

General rule:

Any algorithm that halves the search range at every step needs O(log n) steps.

Specific case:

Binary search halves the search range at every step.

Conclusion:

Therefore, binary search needs O(log n) steps.

B2. Inductive Reasoning

Example: A new sorting algorithm performs better than Quicksort on 100 test cases. Possible inductive conclusion: the new algorithm may perform better in general. The conclusion is not guaranteed.

PRACTICE ZONE — A NEW ML MODEL PERFORMS BETTER ON 8 DATASETS

1. What are the observations?

The new model scored better than the baseline on each of the 8 datasets.

2. What general conclusion might be proposed?

The new model is generally better than the baseline.

3. Why is the conclusion not certain?

Only 8 datasets were tested. A ninth dataset (different size, noise or type) may behave differently, and one counterexample is enough to break a general rule.

B3. Abductive Reasoning

Example: Observation — a program crashes randomly. Possible explanation — a memory leak.

PRACTICE ZONE — A WEB APPLICATION SUDDENLY BECOMES VERY SLOW

Write three possible explanations.

1.

Too many users at the same time (server overload).

2.

Slow database queries, e.g., a missing index causing full table scans.

3.

A recent code update with a bug or a memory leak.

Which explanation would you investigate first?

I would first check the server CPU/memory graphs and logs, and whether a new release went live just before the slowdown. This is quick and rules out overload or a bad update.

PART C — PROOF METHODS

Demonstration, empirical and mathematical approaches

LEARN & REMEMBER

A proof or validation method provides evidence that a research claim is correct, feasible, reliable, or supported.

Method	Basis	Typical use	Main limitation
Demonstration	Examples / prototype	Early validation	Limited cases
Empirical	Experiments / data	Performance studies	Context-dependent
Mathematical	Logic / formal proof	Correctness / theory	Requires formal assumptions

C1. Identify the Method

Write D = Demonstration, E = Empirical, M = Mathematical.

1. Running a prototype on five sample inputs. D
2. Benchmarking an ML model on multiple datasets. E
3. Proving algorithm correctness using formal logic. M
4. Testing network latency under different workloads. E
5. Proving a theorem about computational complexity. M

C2. Choose the Validation Method

Research Claim	Method	Why?
A sorting algorithm is correct	M	Correctness for all inputs needs a formal proof (loop invariant / induction).
A new ML model performs better	E	Performance is measured with metrics on datasets.
A robot prototype can follow a path	D	A working prototype can be shown following test paths.
A cryptographic construction is secure under assumptions	M	Security is proved from stated assumptions (a reduction).

PART D — PROOF BY DEMONSTRATION

Examples and prototypes as early evidence

LEARN & REMEMBER

Proof by demonstration means showing that something works using examples, test cases, or a prototype. It is:

- easy to understand,
- useful for early research,
- useful for prototypes,
- useful for teaching,
- but not a universal proof.

D1. Example

Claim: "This new sorting algorithm is efficient." Suppose we run it on five sample arrays and all five executions are successful. This demonstrates that it works for those tested examples. It does not prove that it is efficient for every possible input.

PRACTICE ZONE — "OUR CHATBOT CORRECTLY HANDLES ALL CUSTOMER QUERIES."

Design:

1. Test case 1:

   A common question: "What are your working hours?"

2. Test case 2:

   An order-status query with a valid order number.

3. Test case 3:

   A question with spelling mistakes: "wher is my ordr".

4. One corner case that might be missed:

   A message with two requests at once ("cancel my order and change my address") or one written in mixed Hindi–English.

D2. Demonstration vs Proof

Demonstration proves that	the method works on the specific cases that were tested.
Demonstration cannot prove that	it is correct or efficient for every possible input.
A better next step could be	testing on large, varied data (empirical study) or giving a formal proof.

PART E — EMPIRICAL PROOFS

Evidence through observation, experimentation and measurement

LEARN & REMEMBER

Empirical validation uses real observations and experimental data. A typical empirical study involves:

Question → Experiment → Measurement → Analysis → Conclusion

Common applications: algorithm benchmarking, ML model evaluation, network performance, usability studies, system experiments.

E1. Example

Claim: "Our new ML model performs better than the baseline." Possible measurements: Accuracy, Precision, Recall, F1-score, Runtime.

PRACTICE ZONE — "A NEW NETWORK PROTOCOL REDUCES LATENCY."

Independent variable	The protocol used (new protocol vs. existing protocol).
Dependent variable	End-to-end latency in milliseconds.
Baseline	The existing standard protocol (TCP).
Dataset / workload	Simulated traffic at low, medium and high load (e.g., in NS3).
Metric	Mean latency and 95th-percentile latency.
Number of trials	30 runs per load level with different random seeds.
Possible limitation	A simulation may not match real network conditions.

E2. Advantages and Limitations

Advantages	**Limitations**
Real-world evidence	Context-dependent
Practical performance	Can be expensive
Supports comparison	External factors matter
Allows iterative improvement	May not generalize

PART F — MATHEMATICAL PROOFS

Rigorous reasoning using logic and mathematics

LEARN & REMEMBER

Mathematical proof provides rigorous validation. Important types: 1. Direct proof 2. Proof by contradiction 3. Proof by induction 4. Proof by contrapositive.

It is widely used in algorithm correctness, complexity, cryptography, automata theory and formal systems.

F1. Direct Proof

A direct proof starts from known assumptions and logically derives the required result.

PRACTICE ZONE — COMPLETE THE STRUCTURE

Statement to prove:

The product of two odd integers is odd.

Given / assumptions:

Let a = 2m + 1 and b = 2n + 1, where m and n are integers.

Logical steps:

ab = (2m + 1)(2n + 1) = 4mn + 2m + 2n + 1

= 2(2mn + m + n) + 1

Conclusion:

ab = 2k + 1 with k = 2mn + m + n (an integer), so ab is odd.

F2. Proof by Contradiction

Basic idea: Assume opposite → derive contradiction → original claim holds. Example in CS: impossibility results such as the Halting Problem can use contradiction-style reasoning.

PRACTICE ZONE — FILL THE STRUCTURE

Claim:

There are infinitely many prime numbers (there is no largest prime).

Opposite assumption:

Assume there are only finitely many primes: p1, p2, ..., pn.

Contradiction:

Let N = p1 × p2 × ... × pn + 1. Dividing N by any pi leaves remainder 1, so no pi divides N.

So N is prime or has a prime factor not in the list — but the list had all primes.

Conclusion:

The assumption is false, so there are infinitely many primes.

F3. Proof by Induction

Induction contains: 1. Base case 2. Inductive hypothesis 3. Inductive step 4. Conclusion. Classic result: 1 + 2 + ... + n = n(n + 1)/2

PRACTICE ZONE

Base case (n = 1):

1 = 1(1 + 1)/2 = 2/2 = 1. Left side equals right side, so the base case is true.

Inductive hypothesis:

Assume 1 + 2 + ... + k = k(k + 1)/2 for some k ≥ 1.

Inductive step: 1 + 2 + ... + k + (k + 1) = k(k + 1)/2 + (k + 1)

Complete: = (k + 1)(___) / 2

Blank = k + 2

k(k + 1)/2 + (k + 1) = (k + 1)(k/2 + 1) = (k + 1)(k + 2)/2

Conclusion:

By induction, 1 + 2 + ... + n = n(n + 1)/2 for all n ≥ 1.

PART G — CONTRAPOSITIVE

A useful form of implication-based reasoning

LEARN & REMEMBER

For an implication P ⇒ Q the contrapositive is ¬Q ⇒ ¬P. An implication and its contrapositive are logically equivalent.

PRACTICE ZONE — CONTRAPOSITIVE OF: "IF AN ALGORITHM TERMINATES, THEN IT PRODUCES AN OUTPUT."

Original statement:

If an algorithm terminates, then it produces an output.

Contrapositive:

If an algorithm does not produce an output, then it does not terminate.

Explain why the two statements are logically related.

They are logically equivalent: whenever one is true the other is true, and whenever one is false the other is false. So proving one proves the other.

G1. Identify the Proof Type

Write: Direct / Contradiction / Induction / Contrapositive

1. Assume the opposite and derive an impossible result. — Contradiction
2. Prove the base case and then k → k + 1. — Induction
3. Prove ¬Q → ¬P to establish P → Q. — Contrapositive
4. Start from known assumptions and derive the result directly. — Direct

G2. Applications

Area	Proof method commonly used
Algorithm correctness	Induction and loop invariants (e.g., proving a sorting algorithm correct).
Complexity analysis	Induction to solve recurrences; contradiction for lower bounds.
Cryptography	Contradiction / reduction (if an attacker succeeds, a hard problem would be broken).
Automata / formal languages	Contradiction (pumping lemma) and induction on string length.

Krish Aggarwal 590014301 B1_CCVT 23/10/2026

PART H — DEDUCTION AND INDUCTION

Using evidence and logic to build CS knowledge

LEARN & REMEMBER

Deduction: General rule → Specific case. If the premises are true and the logic is valid, the conclusion follows.

Induction: Specific observations → General rule. The conclusion is probable rather than guaranteed.

H1. Compare

	Deduction	Induction	Abduction
Starts with	General rule	Observations	Observation
Ends with	Specific conclusion	Generalization	Best explanation
Certainty	High if premises true	Probable	Plausible

H2. Classify the Following

Write D, I or A.

1. A theorem is applied to a particular graph. D
2. A researcher finds the same trend in 100 experiments and proposes a general rule. I
3. A program crashes and the researcher suspects a memory leak. A
4. A formal proof establishes algorithm correctness. D

PRACTICE ZONE — EXPLAIN ONE ANSWER

Choose any one of the above and explain why it is deductive, inductive or abductive.

Item 3 is abductive: from one observation (the crash) the researcher picks the most likely explanation (a memory leak). It is plausible, but it is not certain until it is tested.

PART I — THEORETICAL MODELS

Abstract models for understanding computational systems

LEARN & REMEMBER

Theoretical models provide abstract representations of computational systems. Examples: Finite Automata, Pushdown Automata, Turing Machines, Propositional and Predicate Logic, Graphs and Trees, Boolean Algebra, Formal Grammars.

They help researchers reason about computation without depending completely on a particular physical implementation.

I1. Match the Model

Task	Model
Recognize a formal language	Finite Automaton (regular languages)
Represent a network structure	Graph
Study computability	Turing Machine
Represent Boolean operations	Boolean Algebra
Reason about logical statements	Propositional / Predicate Logic

I2. Turing Machine

A Turing Machine is an abstract computational model used to study computability and decidability.

PRACTICE ZONE — ANSWER BRIEFLY

1. Why can an abstract model be useful?

   It removes small details so we can study the main idea clearly, and its results hold for any hardware.

2. What is meant by computability?

   Whether a problem can be solved by some algorithm (on a Turing Machine) at all.

3. Give one research problem where a theoretical model would be useful.

   Checking whether a language feature, such as matching brackets in code, can be recognised by a pushdown automaton (useful in compiler design).

I3. Advantages and Limitations

Advantages	Limitations
Clarity and precision	May ignore real-world constraints
Formal analysis	Can be highly abstract
Hardware-independent reasoning	May simplify complex systems

PART J — ALGORITHMIC APPROACHES

Designing, analysing and testing computational solutions

LEARN & REMEMBER

Algorithmic research commonly follows: Problem → Algorithm → Complexity → Correctness → Testing

Examples include graph algorithms, sorting, searching, dynamic programming, greedy algorithms and approximation algorithms.

J1. Algorithm Research Canvas

Problem	Searching for a value in a sorted array.
Input	A sorted array A of n numbers and a target value x.
Output	The index of x in A, or −1 if x is not present.
Algorithm idea	Binary search: compare x with the middle element and discard the half that cannot contain x.
Time complexity	O(log n)
Space complexity	O(1) for the iterative version.
Correctness argument	Loop invariant: if x is in A, it lies between low and high. Each step keeps this true and shrinks the range, so the loop ends with the right answer.
Test cases	Empty array; one element; x at the first, middle and last position; x absent; repeated values.

J2. Think

Why should a researcher consider both theoretical complexity and practical execution time?

Big-O shows how running time grows with input size but hides constants, hardware, cache and memory effects. Measured time shows real speed. Using both gives the full picture.

COMMON TRAP

A successful implementation does not automatically prove optimality or correctness for every possible input.

PART K — SOFTWARE ENGINEERING APPROACHES

Applying structured development practices to research

LEARN & REMEMBER

Software engineering approaches provide systematic methods for developing and testing software systems. Common process models: Waterfall, Agile, Spiral.

Simplified research software cycle: Requirements → Design → Implementation → Testing → Maintenance

K1. Research Prototype

Prototype chosen: a spam e-mail classifier.

Requirements	Classify e-mails as spam or not spam with high accuracy and few false alarms.
System design	Modular pipeline: load data → clean text → TF-IDF features → classifier → report.
Implementation	Python with scikit-learn; code kept in a Git repository.
Testing	Unit tests for each module, then evaluation on a held-out test set.
Future enhancement	Add a web interface and retrain regularly on new spam.

K2. Why Software Engineering Matters in Research

Version control	Keeps the history of changes, helps teamwork and allows going back after a mistake.
Testing	Finds bugs early so the results can be trusted.
Documentation	Helps others understand, run and reuse the work.
Reproducibility	Lets others repeat the experiment and get the same result, which makes research believable.

THINK LIKE A RESEARCHER

A research prototype may not become a commercial product. Why, nevertheless, should it be developed systematically?

Research results depend on the software. A systematic prototype is reliable, can be verified by others, and can be reused or extended in later research.

PART L — MATHEMATICAL MODELLING

Representing real systems using mathematical relationships

LEARN & REMEMBER

Mathematical modelling converts a real-world system into a simplified mathematical representation.

System → Assumptions → Variables → Model → Analysis → Validation

Applications: network performance, reliability, queueing systems, graph problems and optimization.

L1. Build a Model

Chosen: IoT System.

System	A battery-powered IoT temperature sensor that wakes up, measures, sends data and sleeps.
Research question	How does the sensing interval (how often it wakes up) affect battery life?
Assumption 1	The sensor uses constant power in the active state and constant (low) power in the sleep state.
Assumption 2	Battery capacity is fixed and does not drop during the test.

Variable 1	E = battery energy (Wh)
Variable 2	d = fraction of time the sensor is active (duty cycle)
Variable 3	Pa and Ps = active power and sleep power (W)
Relationship / equation	Battery life T = E / (d·Pa + (1 − d)·Ps) hours
How will you validate it?	Measure real battery life at different sensing intervals and compare with the model's predictions.

L2. Simple Queueing Example

Packets arrive at a server at rate λ packets/second and are processed at rate μ packets/second.

PRACTICE ZONE

1. What does λ represent?

   The average rate at which packets arrive (packets per second).

2. What does μ represent?

   The average rate at which the server processes packets (packets per second).

3. What would happen if the arrival rate becomes greater than the service rate?

   The queue keeps growing, delay keeps increasing and packets are eventually dropped. The system becomes unstable.

PART M — PERFORMANCE ESTIMATION AND EVALUATION

Measuring efficiency, scalability and reliability

LEARN & REMEMBER

Performance evaluation assesses how effectively a system performs. Important parameters: Time efficiency (execution or response time), Space efficiency (memory/storage), Scalability (behaviour with increasing workload), Reliability (consistency and failure behaviour).

Metric → Experiment → Measurement → Comparison → Interpretation

M1. Evaluation Plan

Research claim	Our spam classifier detects spam more accurately than a baseline Naive Bayes filter.
Metric 1	F1-score (with precision and recall).
Metric 2	Prediction time per e-mail (ms).
Dataset / workload	A public spam e-mail dataset (e.g., Enron-Spam), split into train and test sets.
Baseline	Multinomial Naive Bayes classifier.
Environment	Python 3 and scikit-learn on a laptop (same machine for all runs).
Number of experiments	5-fold cross-validation repeated 10 times.
Possible limitation	The dataset may not represent new spam styles, so results may not generalize.

M2. Example

Model	Accuracy	Precision	Recall	F1
A	94%	91%	88%	89%
B	92%	95%	86%	90%

PRACTICE ZONE

1. Which model has higher accuracy?

Model A (94% vs 92%).

2. Which model has higher precision?

Model B (95% vs 91%).

3. Which model has higher F1-score?

Model B (90% vs 89%).

4. Can you declare one model universally “better” using only these values? Why?

No. A is better in accuracy and recall, B is better in precision and F1. The better choice depends on the goal (for example, fewer false alarms favours B, catching more positives favours A) and on the data used.

PART N — PERFORMANCE TOOLS

Selecting appropriate tools for research evaluation

LEARN & REMEMBER

Tool categories used for performance studies: Profilers, Simulation tools, Benchmarking suites, Monitoring tools.

Examples from the Unit II material: gprof, Valgrind, JProfiler, NS3, CloudSim, OMNeT++, SPEC CPU, MLPerf, Prometheus and Grafana.

PRACTICE ZONE — MATCH THE REQUIREMENT WITH AN APPROPRIATE TOOL/CATEGORY

1. Measure CPU and memory behaviour of a program.

Profiler (gprof / Valgrind / JProfiler).

2. Simulate network behaviour.

Network simulator (NS3 / OMNeT++).

3. Compare standardized computing performance.

Benchmarking suite (SPEC CPU / MLPerf).

4. Monitor system metrics continuously.

Monitoring tools (Prometheus / Grafana).

N1. Researcher’s Performance Sheet

Experiment title	Comparing a new spam classifier with a Naive Bayes baseline.
Hypothesis	The new classifier gets a higher F1-score than the baseline on the same data.
Metric(s)	F1-score, precision, recall and prediction time.
Observed result	Example values for practice: F1 = 0.96 (new) vs 0.93 (baseline).
Interpretation	On this dataset the new classifier detects spam better than the baseline.
Limitation	Only one dataset and a small gain; more datasets and a significance test are needed.

PART O — INTEGRATED RESEARCH ACTIVITY

Apply multiple Unit II concepts to one research problem

CLASSROOM ACTIVITY — SCENARIO

A team proposes: “Our AI-enabled smart irrigation system reduces water consumption while maintaining crop health.” You are the research-methods investigator. Your task is to examine the claim scientifically.

O1. Dialectical Analysis

Original claim	The AI irrigation system cuts water use while keeping crops healthy.
Counter-argument 1	The water saving may come from a rainy season, not from the AI.
Counter-argument 2	Faulty or dirty sensors could give wrong readings and under-water the crops.
Refined claim	In field trials on the tested crop and soil, the AI system uses less water than fixed-schedule irrigation while crop yield and health stay about the same.

O2. Argument Model

Which reasoning model would be appropriate?

Using observations to generalize performance	Inductive
Applying a known theorem to the system	Deductive
Finding the most likely cause of a sensor problem	Abductive

O3. Proof / Validation

Select appropriate methods:

☑ Demonstration ☑ Empirical ☐ Mathematical

Explain your choice:

A demonstration on a small plot first shows the prototype works. The real proof is an empirical field experiment that measures water used and crop health over a season. A mathematical proof is not needed because this is a performance claim, not a correctness theorem.

O4. Algorithmic Approach

What algorithm could decide when irrigation should start?	A rule-based (threshold) algorithm: if soil moisture is below the crop’s limit and no rain is forecast, start irrigation.
What would be its input?	Soil-moisture reading, rain forecast, crop type / growth stage.
What would be its output?	Irrigation ON/OFF and for how long.

O5. Mathematical Model

x1 =	Soil moisture (available water, in mm)
x2 =	Forecast rainfall (mm)
x3 =	Crop water requirement (mm per day)
Assumption	Sensors are accurate and the crop need is roughly constant within a day.
Simple relationship / model	Water to apply W = max(0, x3 − x1 − x2)

O6. Performance Evaluation

Metric 1	Total water used (litres per hectare).
Metric 2	Crop yield / crop-health index.
Baseline	Fixed-schedule (timer-based) irrigation.

Experiment condition	Two identical plots (same crop, soil, season): one with the AI system, one with the baseline.
Expected result	The AI plot uses less water with yield close to the baseline plot.

PART P — MINI RESEARCH DESIGN

Create your own small Computer Science research study

Research Title	Improving Spam E-mail Detection Using TF-IDF Features and a Linear SVM
Research problem	Simple spam filters miss some spam and sometimes block genuine mail.
Research question	Can a TF-IDF + linear SVM classifier detect spam with a higher F1-score than a Naive Bayes baseline?
Claim / hypothesis	TF-IDF + linear SVM gives a higher F1-score than Naive Bayes on the same dataset.
Counter-argument	Naive Bayes is simpler and faster; the SVM gain may be tiny and may not hold for new spam styles.
Reasoning model	Inductive (generalising from results over many data splits).
Validation / proof method	Empirical evaluation with cross-validation.
Theoretical model	Vector-space model of text; a linear separating hyperplane.
Algorithmic approach	Clean text → TF-IDF → train linear SVM → classify.
Mathematical model	P = TP/(TP+FP), R = TP/(TP+FN), F1 = 2PR/(P+R)
Performance metric	F1-score, precision, recall and prediction time.
Baseline	Multinomial Naive Bayes.
Expected contribution	A simple, reproducible spam filter with better detection and fewer false alarms.

THINK LIKE A RESEARCHER — RESEARCHER'S CHECK

Before accepting your own claim, ask:

1. What could prove me wrong? 2. What evidence do I need? 3. Which assumptions am I making? 4. Can another researcher reproduce my experiment? 5. Does my evidence actually support my conclusion?

Research Question	→ Evidence	→ Model / Method	→ Evaluation
Is the SVM better than Naive Bayes?	F1 scores from repeated runs	TF-IDF + linear SVM	Compare F1 with baseline

UNIT II COMPLETE

Think critically. Question assumptions. Use evidence. Build arguments. Validate claims. Evaluate results.

Krish Aggarwal 590014301 B1_CCVT 23/10/2026